

Networking & Cybersecurity Track
Simulated Employment Program

Lab Report:
Navigating the Linux Filesystems and Permissions Settings

Prepared by:
Kudzaishe Majeza
Junior Network Engineer – VxLEARN Networks

Mentor:
Titus Majeza
Senior Network Engineer

Date: 20 April 2026

Objective

This lab develops practical Linux system administration skills across three core areas. Part 1 covers block device identification and file system mounting using lsblk and mount. Part 2 explores the Linux permission model: reading ls -l output, correcting access with chmod, and reassigning ownership with chown. Part 3 examines the difference between symbolic links and hard links through direct experimentation, including how each responds to renaming the original file (Cisco Networking Academy, 2025).

Part 1: Exploring Filesystems in Linux

1.1 Background — Block Devices and Disk Naming

In Linux, every storage device is represented as a file inside the /dev/ directory. Physical and virtual disks are called block devices because they transfer data in fixed-size chunks (The Linux Kernel Organization, 2025). Linux uses a predictable naming scheme: the first disk is /dev/sda, the second /dev/sdb, and so on. Partitions on a disk are numbered: /dev/sda1, /dev/sda2. Optical drives are labelled /dev/sr0. Understanding this naming convention is essential before mounting or partitioning any storage (The Linux Man Pages Project, 2025a).

1.2 Step 1 — Listing Block Devices with lsblk

The lsblk command ("list block devices") produces a tree view of every block device:

```
[analyst@secOps ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sda          8:0    0   20G  0 disk
└─sda1       8:1    0   20G  0 part /
sdb          8:16   0    4G  0 disk
└─sdb1       8:17   0    4G  0 part
sr0         11:0    1 1024M  0 rom
```

sda1 is the system partition mounted at / (the Linux root). sdb and sdb1 show no MOUNTPOINT — they are attached but not yet accessible. sr0 is the virtual optical drive (Nguyen, 2004).

Column	Meaning
NAME	Device name: sda = first disk, sda1 = first partition, sr0 = optical
MAJ:MIN	Major:Minor device numbers used by the kernel to identify the driver
SIZE	Total storage capacity of the disk or partition
TYPE	disk = whole drive part = partition rom = optical media
MOUNTPOINT	Directory path where the filesystem is accessible. Blank = not mounted

1.3 Step 2 — Creating a Mount Point and Mounting /dev/sdb1

Before a file system can be accessed, a mount point directory must exist. A mount point is simply an empty directory that becomes the entry point into the mounted file system — analogous to a drive letter (e.g., D:) in Windows (Red Hat, Inc., 2022).

```
# Step 1: Create the mount point directory
[analyst@secOps ~]$ sudo mkdir /mnt/second_drive

# Step 2: Mount the partition to the directory
[analyst@secOps ~]$ sudo mount /dev/sdb1 /mnt/second_drive

# Step 3: Verify using lsblk
[analyst@secOps ~]$ lsblk
`-sdb1   8:17   0   4G  0 part /mnt/second_drive

# Step 4: Browse the mounted filesystem
[analyst@secOps ~]$ ls /mnt/second_drive
lost+found
```

The lost+found directory is automatically created by ext-family file systems. It serves as a recovery location for the fsck (file system check) utility to place orphaned files after an unclean shutdown (The Linux Kernel Organization, 2025).

Key concept: mount connects a block device to a directory. umount disconnects it. The directory still exists after unmounting — only the connection is removed.

1.4 Step 3 — Unmounting with umount

When finished, unmount the file system to flush pending writes and safely detach the partition. Note the spelling: umount, not "unmount".

```
[analyst@secOps ~]$ sudo umount /mnt/second_drive
[analyst@secOps ~]$ lsblk
`-sdb1   8:17   0   4G  0 part    <- MOUNTPOINT is blank again
```

Part 2: File Permissions, chmod, and chown

2.1 Background — The Linux Permission Model

Linux enforces access control through permission bits embedded in every file and directory. Each file has three permission sets: Owner (the user who created or owns the file), Group (a named collection of users), and Others (everyone else on the system). Each set can grant three rights: read (r=4), write (w=2), and execute (x=1). These map directly to a three-bit binary value, which is commonly expressed as a single octal digit 0–7 (GNU Project, 2025a).

The `ls -l` command displays a ten-character string for each file: the first character is the file type, followed by three triplets of rwx bits for Owner, Group, and Others respectively.

```
[analyst@secOps ~]$ ls -la ~/scripts/
-rwxr-xr-x 1 root root 253 Jun 5 12:00 cyberops_extended.sh
-rwxr-xr-x 1 root root 128 Jun 5 12:00 myscript.py

# Breakdown of -rwxr-xr-x:
# - = regular file (type character)
# rwx = Owner (root): Read + Write + Execute
# r-x = Group: Read + Execute only (no write)
# r-x = Others: Read + Execute only (no write)
```

The analyst user has no write access here because they are not the owner (root) and the Group and Others triplets do not include w (Shields, 2016).

2.2 Octal Permission Reference

Octal	Binary	Permissions	Meaning
7	111	rwx	Read + Write + Execute (full access)
6	110	rw-	Read + Write
5	101	r-x	Read + Execute
4	100	r--	Read only
3	011	-wx	Write + Execute
2	010	-w-	Write only
1	001	--x	Execute only
0	000	---	No permissions at all

2.3 Step 1 — Permission Denied Error

When analyst attempts to create a file in a root-owned directory:

```
[analyst@secOps ~]$ touch ~/scripts/newfile.txt
touch: cannot touch newfile.txt: Permission denied
```

This error occurs because the scripts/ directory is owned by root and the Others triplet does not include w. Without write permission on the directory, no new files can be created inside it by anyone other than root (GNU Project, 2025a).

2.4 Step 2 — Changing Permissions with chmod

chmod (change mode) modifies permission bits. Using octal notation, chmod 665 sets: Owner = rw-(6), Group = rw-(6), Others = r-x (5).

```
# Apply chmod 665: Owner=rw-(6), Group=rw-(6), Others=r-x(5)
[analyst@secOps ~]$ sudo chmod 665 ~/scripts/

# Verify the new permissions
[analyst@secOps ~]$ ls -la ~/
drwxrw-r-x 2 root root 4096 Jun 10 09:10 scripts/
# d = directory rw- = owner rw- = group r-x = others
```

chmod 665 adds group write permission while keeping Others at read+execute. The octal calculation: $r=4 + w=2 + 0 = 6$ (rw-), and $r=4 + 0 + x=1 = 5$ (r-x) (GNU Project, 2025b).

2.5 Step 3 — Changing Ownership with chown

chown (change owner) transfers file ownership. Only root or a sudo user can reassign ownership to a different account:

```
[analyst@secOps ~]$ sudo chown analyst ~/scripts/

# Verify the new owner
[analyst@secOps ~]$ ls -la ~/
drwxrwx-r-x  2  analyst  analyst  4096  Jun 10 09:15  scripts/

# Now analyst can create and write files
[analyst@secOps ~]$ touch ~/scripts/newfile.txt      # Success!
[analyst@secOps ~]$ echo "Hello lab" >> ~/scripts/newfile.txt
```

Security note: The root user bypasses ALL permission checks and can read, write, or execute any file regardless of permission bits. Protecting the root account with a strong password is therefore essential (Cisco Networking Academy, 2025).

Part 3: Symbolic Links and Special File Types

3.1 Linux File Types — The First Character of ls -l

The very first character of every ls -l line identifies the file type. Linux recognises three main categories: regular files, directory files, and special files. Each has a specific character code:

Char	Category	Type	Description
-	Regular File	Text / Binary / Image / Compressed	Most common. Includes readable text, compiled programs, images, and archives.
d	Directory File	Folder	A special file containing a table of filenames mapped to inodes.
b	Special File	Block Device	Hardware accessed in fixed-size blocks. e.g. /dev/sda, /dev/sdb1 (hard drives).
c	Special File	Character Device	Hardware accessed one character at a time. e.g. /dev/tty (terminal).
p	Special File	Pipe (FIFO)	Passes data between processes. First bytes in = first bytes out.
l	Special File	Symbolic Link	Stores a path pointing to another file or directory (like a Windows

			shortcut).
s	Special File	Socket	Passes data between applications over a network or locally.

3.2 Step 1a — Examining File Types in the Home Directory

Use `ls -l` on the `/home/analyst` directory. Notice the first character of each line: a dash (-) indicates a regular file and a `d` indicates a directory.

```
[analyst@secOps ~]$ ls -l

total 28
drwxr-xr-x 3 analyst analyst 4096 Aug 16 15:15 cyops_folder2
drwxr-xr-x 2 analyst analyst 4096 Sep 26 2014 Desktop
drwx----- 3 analyst analyst 4096 Jul 14 11:28 Downloads
drwxr-xr-x 9 analyst analyst 4096 Jul 25 16:27 lab.support.files
drwxr-xr-x 3 analyst analyst 4096 Mar 3 18:23 second_drive
-rw-r--r-- 1 analyst analyst 142 Aug 16 15:11 some_text_file.txt
-rw-r--r-- 1 analyst analyst 254 Aug 16 13:38 space.txt
```

Every line beginning with `d` is a directory: `cyops_folder2`, `Desktop`, `Downloads`, `lab.support.files`, and `second_drive`. Every line beginning with `-` is a regular file: `some_text_file.txt` and `space.txt`. The size shown for directories (4096) is the size of the directory file itself, not the combined size of its contents.

3.3 Step 1b — Viewing Block, Character, and Link Files in `/dev/`

Run `ls -l` on the `/dev/` directory. This is where Linux stores all device files. Scroll through the output and observe the `b` (block), `c` (character), and `l` (symbolic link) type characters:

```
[analyst@secOps ~]$ ls -l /dev/

crw-rw-rw- 1 root tty      5,  2 May 29 18:32 ptmx
drwxr-xr-x 2 root root      0    May 23 06:40 pts
crw-rw-rw- 1 root root      1,  8 May 23 06:41 random
crw-rw-r-- 1 root root    10, 56 May 23 06:41 rfcill
lrwxrwxrwx 1 root root      4    May 23 06:41 rtc -> rtc0
crw-rw---- 1 root audio 253,  0 May 23 06:41 rtc0
brw-rw---- 1 root disk    8,  0 May 23 06:41 sda
brw-rw---- 1 root disk    8,  1 May 23 06:41 sda1
brw-rw---- 1 root disk    8, 16 May 23 06:41 sdb
brw-rw---- 1 root disk    8, 17 May 23 06:41 sdb1
drwxrwxrwt 2 root root    40    May 28 13:47 shm
crw----- 1 root root   10,231 May 23 06:41 snapshot
brw-rw----+ 1 root optical 11,  0 May 23 06:41 sr0
lrwxrwxrwx 1 root root    15    May 23 06:40 stderr -> /proc/self/fd/2
lrwxrwxrwx 1 root root    15    May 23 06:40 stdin  -> /proc/self/fd/0
lrwxrwxrwx 1 root root    15    May 23 06:40 stdout -> /proc/self/fd/1
crw-rw-rw- 1 root tty      5,  0 May 29 17:36 tty
crw--w---- 1 root tty      4,  0 May 23 06:41 tty0
```

Type char	Examples from output	What they are
b Block device	sda, sda1, sdb, sdb1, sr0	Physical disks and partitions. sda = first hard drive. sda1 = its first partition (the one holding the OS). sr0 = optical drive. These are the same devices we mounted in Part 1.
c Character device	ptmx, random, rfkill, tty, tty0	Hardware accessed byte-by-byte. random generates random numbers from hardware noise. tty and tty0 are terminal interfaces. ptmx is the pseudoterminal multiplexer.
l Symbolic link	rtc -> rtc0 stderr -> /proc/self/fd/2 stdin -> /proc/self/fd/0 stdout -> /proc/self/fd/1	The arrow (->) shows exactly where each symlink points. rtc is a shortcut to rtc0. stdin, stdout, and stderr are the three standard I/O streams every Linux process uses, implemented as symlinks into /proc/self/fd/.

3.4 Step 1c — Creating the Test Files

Symbolic links in Linux are like shortcuts in Windows. There are two types: symbolic links and hard links. The difference is that a symbolic link points to the filename of another file, while a hard link points to the contents (inode) of another file. Create two test files using echo and verify their contents with cat:

```
[analyst@secOps ~]$ echo 'symbolic' > file1.txt
[analyst@secOps ~]$ cat file1.txt
symbolic

[analyst@secOps ~]$ echo 'hard' > file2.txt
[analyst@secOps ~]$ cat file2.txt
hard
```

The echo command writes text to a file using the > redirect operator. file1.txt contains the word "symbolic" and file2.txt contains "hard". cat reads and prints the file contents to confirm the writes succeeded.

3.5 Step 1d — Creating a Symbolic Link and a Hard Link

Every file on disk has an inode — a data structure storing the file's metadata (permissions, owner, size, timestamps) and pointers to the actual data blocks. The filename in a directory is just a label mapped to an inode number. A hard link adds a second label pointing to the same inode. A symbolic link is its own file whose content is a path string to the target.

Use ln -s to create a symbolic link to file1.txt, and ln to create a hard link to file2.txt:

```
# Symbolic link: file1symbolic stores the path "file1.txt"
[analyst@secOps ~]$ ln -s file1.txt file1symbolic
```

```
# Hard link: file2hard points directly to the same inode as file2.txt
[analyst@secOps ~]$ ln file2.txt file2hard
```

ln -s creates a symbolic (soft) link. ln with no flag creates a hard link. No output from either command means both succeeded.

3.6 Step 1e — Inspecting the Links with ls -l

Run `ls -l` and examine the full directory listing:

```
[analyst@secOps ~]$ ls -l

total 40
drwxr-xr-x 3 analyst analyst 4096 Aug 16 15:15 cyops_folder2
drwxr-xr-x 2 analyst analyst 4096 Sep 26 2014 Desktop
drwx----- 3 analyst analyst 4096 Jul 14 11:28 Downloads
lrwxrwxrwx 1 analyst analyst  9 Aug 17 16:43 file1symbolic -> file1.txt
-rw-r--r-- 1 analyst analyst  9 Aug 17 16:41 file1.txt
-rw-r--r-- 2 analyst analyst  5 Aug 17 16:42 file2hard
-rw-r--r-- 2 analyst analyst  5 Aug 17 16:42 file2.txt
drwxr-xr-x 9 analyst analyst 4096 Jul 25 16:27 lab.support.files
drwxr-xr-x 3 analyst analyst 4096 Mar  3 18:23 second_drive
-rw-r--r-- 1 analyst analyst 142 Aug 16 15:11 some_text_file.txt
-rw-r--r-- 1 analyst analyst 254 Aug 16 13:38 space.txt
```

Entry	What to notice	Why
file1symbolic -> file1.txt	Type character = l Arrow -> shows the target	l confirms this is a symbolic link. The arrow tells you exactly where it points. Size = 9, which is the number of characters in the string "file1.txt" stored inside it. Link count = 1 (its own inode, referenced by only this one name).
file2hard and file2.txt	Both show type - Both show link count = 2 Same size (5) and timestamp	Hard links look like normal files because they ARE normal directory entries. The link count of 2 means two names (file2.txt and file2hard) point to the same inode. Identical size and timestamp confirm they share one inode and one set of data blocks.

For directories, the second column (link count) shows the number of subdirectories inside, including the hidden `.` and `..` entries. For files, it is always the hard link count.

3.7 Step 1f — Rename the Originals and Test Both Links

Change the names of `file1.txt` and `file2.txt` using `mv`, then test both links. `mv` only updates the directory entry (the label). It does not touch the inode or data blocks.

```
[analyst@secOps ~]$ mv file1.txt file1new.txt
```



```
[analyst@secOps ~]$ mv file2.txt file2new.txt
```

Now test the symbolic link:

```
[analyst@secOps ~]$ cat file1symbolic
cat: file1symbolic: No such file or directory
```

Now test the hard link:

```
[analyst@secOps ~]$ cat file2hard
hard
```

Link	Result after mv	Reason
file1symbolic (symbolic link)	BROKEN ✗ "No such file or directory"	file1symbolic stores the path string "file1.txt". After mv, that name no longer exists in the directory. Linux follows the path, finds nothing, and fails. This is called a dangling symlink.
file2hard (hard link)	WORKS ✓ prints "hard" correctly	file2hard points to an inode number, not a name. mv renamed file2.txt to file2new.txt but never changed the inode. The inode still exists with the same data, so file2hard reads it perfectly.

3.8 Reflection Question

The lab asks: What would happen to file2hard if you opened a text editor and changed the content in file2new.txt?

Answer: file2hard would immediately reflect every change. file2new.txt and file2hard both point to the same inode, which points to the same data blocks on disk. There is only one copy of the data. Editing through file2new.txt writes directly to those data blocks. Reading through file2hard reads from those exact same blocks. They are not copies that sync — they are the same file accessed through two different names.

3.9 Hard Link vs Symbolic Link — Full Comparison

Property	Hard Link (ln)	Symbolic Link (ln -s)
Lab command	ln file2.txt file2hard	ln -s file1.txt file1symbolic
ls -l type character	- (identical to a regular file)	l with arrow: file1symbolic -> file1.txt
Inode number	Same inode as original file	Own separate inode
Link count in ls -l	2 (two names, one inode)	1 (independent inode)
Size shown in ls -l	Same as original (actual data size)	Length of the path string stored inside
After mv of original	WORKS ✓ inode is unchanged by mv	BROKEN ✗ stored name no longer exists
After rm of original	Data survives until link count = 0	BROKEN ✗ dangling symlink
Cross filesystems?	No ✗ same filesystem only	Yes ✓ can link across

		filesystems
Link to directories?	No ✗ restricted by OS	Yes ✓ fully supported
What it stores	Direct pointer to inode on disk	Path string to the target file

3.10 Reflection

File permissions and ownership are two of the most important aspects of Linux, and they are also a common cause of problems. A file with wrong permissions or wrong ownership will not be available to the programs that need to access it — the program will usually break and errors will be encountered.

Understanding file types, permissions, and link behaviour is foundational for cybersecurity. Attackers who gain access to a Linux system often exploit misconfigured permissions or create symlinks that redirect programs to malicious targets. Knowing how these mechanisms work at the inode level is what allows a security professional to detect, understand, and fix such attacks.

Conclusion

This lab provided hands-on experience with three foundational pillars of Linux administration. In Part 1, the `lsblk` and `mount/umount` commands demonstrated how Linux identifies block devices and makes their file systems accessible through mount points — a mechanism that underpins all file access at the OS level. In Part 2, the `ls -l` permission string was decoded and corrected using `chmod` (to change permission bits) and `chown` (to transfer ownership), addressing the real-world scenario of a non-root user being blocked from writing to a root-owned directory. Finally, Part 3 revealed through direct experimentation that hard links are robust to file renames because they reference inodes directly, while symbolic links are flexible but fragile because they reference file names that can change or disappear.

Together, these skills form the practical foundation for Linux security work: understanding who can access what, how storage is organised and connected, and how file references behave across operations like rename and delete is essential for any security or systems administration role.

References

1. Cisco Networking Academy. (2025). *Endpoint Security: Lab — Navigating the Linux Filesystem and Permission Settings*. Cisco Systems. <https://www.netacad.com/courses/endpoint-security>
2. GNU Project. (2025a). *File permissions (GNU Coreutils 9.10)*. Free Software Foundation. https://www.gnu.org/software/coreutils/manual/html_node/File-permissions.html

3. GNU Project. (2025b). *chmod invocation (GNU Coreutils 9.10)*. Free Software Foundation.
https://www.gnu.org/software/coreutils/manual/html_node/chmod-invocation.html
4. The Linux Kernel Organization. (2025). *Filesystems in the Linux kernel*.
<https://docs.kernel.org/filesystems/index.html>
5. The Linux Man Pages Project. (2025a). *lsblk(8) — Linux manual page*. <https://man7.org/linux/man-pages/man8/lsblk.8.html>
6. The Linux Man Pages Project. (2025b). *ln(1) — Linux manual page*. <https://man7.org/linux/man-pages/man1/ln.1.html>
7. Nguyen, B. (2004). *Linux filesystem hierarchy (Version 0.65)*. The Linux Documentation Project.
<https://tldp.org/LDP/Linux-Filesystem-Hierarchy/html/index.html>
8. Red Hat, Inc. (2022). *Managing file systems*. Red Hat Enterprise Linux 9.
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/managing_file_systems/index
9. Shields, I. (2016, January 27). *Prepare for LPIC-1 exam 1 — topic 104.5: Manage file permissions and ownership*. IBM Developer. <https://developer.ibm.com/tutorials/l-lpic1-104-5/>